Programação Linear Inteira — Projeto de Implementação

Segundo Semestre de 2026

Prof. Flávio K. Miyazawa, fkm@ic.unicamp.br

Problema de Bin Packing com Classes de Conflito

1 Contexto e visão geral

Uma empresa precisa guardar (ou transportar) vários itens em recipientes idênticos, todos com a mesma capacidade. Ela quer usar o *menor número possível de recipientes*. Esse é o problema clássico de *bin packing*. Neste projeto, há um ingrediente a mais:

- **Classes de conflito.** Alguns itens não podem ficar no mesmo recipiente (por exemplo, alimentos e produtos químicos, ou itens de fornecedores concorrentes). Isso é descrito por *classes de conflito*: cada classe é um conjunto de itens, e *dois itens de uma mesma classe não podem ser colocados no mesmo recipiente*. Itens de classes diferentes podem ficar juntos. Um item pode pertencer a várias classes, ou a nenhuma.

Há uma tensão entre os dois objetivos. A capacidade pede que os recipientes sejam bem cheios, enquanto as classes de conflito obrigam a separar itens, o que pode deixar espaço vazio. O problema consiste em encontrar o melhor compromisso, isto é, empacotar todos os itens no menor número de recipientes.

2 Notação

- Se f é uma função definida sobre um conjunto finito S e $B \subseteq S$, escrevemos $f(B) = \sum_{e \in B} f(e)$. Em particular, para os tamanhos s, temos $s(B) = \sum_{i \in B} s_i$.
- $\mathbb{N}^* = \{1, 2, 3, \ldots\}$ é o conjunto dos inteiros positivos.
- $\lceil x \rceil$ é o menor inteiro maior ou igual a x.

3 Dados de entrada

Uma instância do problema é dada por:

- um inteiro $n \geq 1$ e o conjunto de *itens* $I = \{1, 2, \ldots, n\}$;
- um inteiro $C \in \mathbb{N}^*$, a *capacidade* de cada recipiente;
- um *tamanho* inteiro s_i para cada item $i \in I$, com $0 < s_i \leq C$;
- um inteiro $k \geq 0$ e k subconjuntos de itens $S_1, S_2, \ldots, S_k \subseteq I$, as *classes de conflito*.

As classes não precisam ser disjuntas, não precisam cobrir I, e podem ter qualquer tamanho. Uma classe com no máximo um item não impõe restrição alguma.

4 Recipientes válidos

Definição 1. Um subconjunto $B \subseteq I$ de itens é um *recipiente válido* se

(a) $s(B) \leq C$ (*capacidade*); e

(b) $|B \cap S_t| \leq 1$ para todo $t \in \{1, \ldots, k\}$ (*conflitos*): dois itens $i \neq j$ de B nunca pertencem a uma mesma classe S_t.

5 Soluções viáveis

Definição 2. Uma *solução viável* é uma coleção $\mathcal{B} = \{B_1, B_2, \ldots, B_m\}$ de recipientes válidos que forma uma *partição* de I: os conjuntos B_j são disjuntos dois a dois, e $B_1 \cup \cdots \cup B_m = I$. O *valor* de $\mathcal{B}$ é o número $m = |\mathcal{B}|$ de recipientes usados.

6 O problema

Bin Packing com Classes de Conflito. Dados I, C, s e $S_1, \ldots, S_k$ como na Seção 3, encontrar uma solução viável $\mathcal{B}$ de *menor valor* $|\mathcal{B}|$.

Denotamos por OPT o menor valor de uma solução viável.

Fato 1. (a) $\mathcal{B} = \{\{1\}, \{2\}, \ldots, \{n\}\}$ é uma solução viável. Portanto, toda instância tem solução e $\text{OPT} \leq n$.

(b) $\text{OPT} \geq \lceil s(I)/C \rceil$.

(c) $\text{OPT} \geq |S_t|$ para toda classe S_t.

Demonstração. (a) Cada conjunto $\{i\}$ é um recipiente válido, pois $s_i \leq C$ e um recipiente com um único item não tem dois itens em conflito. (b) Em uma solução com m recipientes, $s(I) = \sum_j s(B_j) \leq mC$. (c) Os itens de S_t não podem ser colocados dois a dois no mesmo recipiente, de modo que ficam em $|S_t|$ recipientes distintos. □

Fato 2. O problema é $\mathcal{NP}$-difícil, mesmo quando $k = 0$.

Demonstração. Com $k = 0$ não há conflitos, e o problema é o *bin packing* clássico, que é $\mathcal{NP}$-difícil (Garey e Johnson). □

7 Um exemplo completo

Considere $n = 8$ itens, capacidade $C = 10$ e $k = 3$ classes de conflito:

$$S_1 = \{1, 2\}, \qquad S_2 = \{3, 4\}, \qquad S_3 = \{5, 6, 7\}.$$

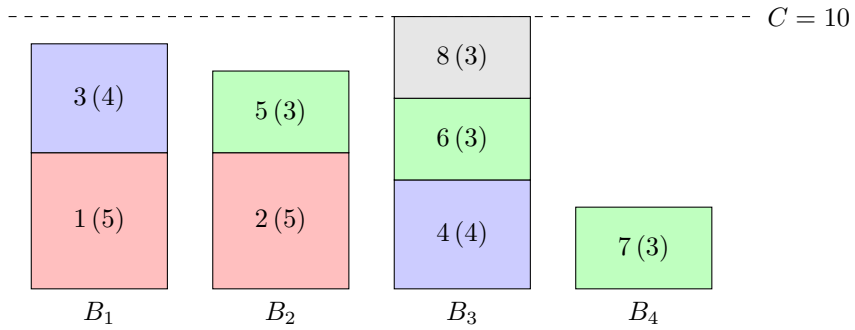
Os tamanhos são:

item i	1	2	3	4	5	6	7	8
tamanho s_i	5	5	4	4	3	3	3	3

O tamanho total é $s(I) = 30$, e as cotas do Fato 1 dão $\lceil 30/10 \rceil = 3$ e $\max_t |S_t| = 3$. Mesmo assim, três recipientes não bastam. Uma solução viável com quatro recipientes é

$$B_1 = \{1, 3\}, \quad B_2 = \{2, 5\}, \quad B_3 = \{4, 6, 8\}, \quad B_4 = \{7\},$$

com cargas $s(B_1) = 9$, $s(B_2) = 8$, $s(B_3) = 10$ e $s(B_4) = 3$. Ela é viável: nenhum recipiente excede $C = 10$, e nenhum deles contém dois itens de uma mesma classe (por exemplo, B_1 tem o item 1 de S_1 e o item 3 de S_2).



Cada retângulo é um item, rotulado por $i\,(s_i)$. Vermelho: S_1; azul: S_2; verde: S_3; cinza: nenhuma classe.

Proposição 3. O valor ótimo da instância acima é OPT = 4.

Demonstração. A solução acima mostra que OPT ≤ 4. Suponha, por absurdo, que existe uma solução viável com 3 recipientes. Como $s(I) = 30 = 3C$ e cada recipiente tem carga no máximo 10, todos os três têm carga *exatamente* 10. O recipiente que contém o item 1 (de tamanho 5) precisa então de outros itens de tamanho total exatamente 5. O item 2 não pode ser usado, pois $1, 2 \in S_1$. Os demais itens têm tamanhos $4, 4, 3, 3, 3, 3$, e nenhum subconjunto deles soma 5. Uma contradição. □

8 Observações para pensar

- **Grafo de conflitos.** Seja $G = (I, E)$ o grafo cujas arestas são os pares $\{i, j\}$, $i \neq j$, que pertencem a alguma classe comum. Então um conjunto B é um recipiente válido se, e somente se, é um conjunto *independente* de G com $s(B) \leq C$. Cada classe é uma *clique* de G, de modo que as classes são uma representação compacta de G quando ele é formado por cliques. Qualquer grafo pode ser descrito com uma classe de dois itens por aresta.
- **Classes grandes ajudam.** Se uma classe S_t tem $|S_t|$ itens, então OPT $\geq |S_t|$ (Fato 1). Em instâncias com classes grandes, ela pode ser maior que a cota de volume $\lceil s(I)/C \rceil$.
- **Sem conflitos, o problema é o bin packing.** Algoritmos conhecidos para o bin packing (por exemplo, *First Fit Decreasing*) precisam ser adaptados para evitar colocar dois itens de uma mesma classe no mesmo recipiente.
- **Simetria.** Os recipientes são idênticos: trocar a ordem dos recipientes de uma solução não muda a solução nem o seu valor.

9 Convenções e formato dos arquivos

9.1 Convenções

- Os itens são numerados $1, 2, \ldots, n$ (e não a partir de 0), tanto nos arquivos quanto na Seção 3.
- As classes são $S_1, \ldots, S_k$, na ordem em que aparecem no arquivo.
- Se um item aparece mais de uma vez na descrição de uma mesma classe, as repetições são ignoradas.

9.2 Arquivo de entrada

O arquivo de entrada é um arquivo texto. Linhas que começam com o caractere `#` são comentários e devem ser ignoradas. As demais linhas aparecem nesta ordem:

1. **Uma linha** com n, C e k: `<n> <C> <k>`.

2. **Uma linha** com os n tamanhos, em ordem crescente do item:

```
<s_1> <s_2> ... <s_n>
```

3. k **linhas**, uma para cada classe S_t, $t = 1, \ldots, k$:

```
<m_t> <i_1> <i_2> ... <i_m_t>
```

onde m_t é o número de itens da classe e $i_1, \ldots, i_{m_t}$ são os seus itens.

Exemplo. A instância da Seção 7 corresponde ao arquivo abaixo.

```
# n (itens), C (capacidade), k (classes de conflito)
8 10 3
# tamanhos s_1 ... s_n
5 5 4 4 3 3 3 3
# classes: m_t seguido dos itens da classe
2 1 2
2 3 4
3 5 6 7
```

9.3 Arquivo de saída

O programa grava a solução encontrada em um arquivo texto, também com comentários iniciados por #. Ele contém, nesta ordem:

1. o valor $|\mathcal{B}|$ da melhor solução encontrada;
2. o melhor limitante inferior para OPT conhecido pelo algoritmo;
3. o número m de recipientes;
4. m linhas: a j-ésima linha contém os itens do recipiente B_j, em qualquer ordem.

Para a instância do exemplo, uma saída com a solução ótima é:

```
# valor da solucao (numero de recipientes)
4
# limitante inferior
4
# numero de recipientes
4
# itens de cada recipiente
1 3
2 5
4 6 8
7
```

10 Projeto 2: Algoritmo *Branch-and-Price*

10.1 O que deve ser feito

Neste segundo projeto, vocês devem implementar um algoritmo exato de *branch-and-price* para o Problema de Bin Packing com Classes de Conflito, partindo do código fornecido (Seção 10.2). Os requisitos são:

1. **Formulação.** Usar necessariamente variáveis binárias x_B, uma para cada recipiente válido B (Seção 10.3).

2. **Geração de colunas.** O problema mestre deve ser resolvido por geração de colunas (Seção 10.4). Cada coluna nova deve ser obtida assim:

   - *primeiro*, por um método heurístico;
   - *somente se* a heurística não encontrar nenhuma coluna candidata, por um método exato. Ele pode ser um método de enumeração, uma formulação de programação linear inteira, programação dinâmica, ou uma combinação deles.

3. **Ramificação.** A árvore de enumeração deve usar o *branching de Ryan–Foster*, visto em sala (Seção 10.5).

4. **Uso do Gurobi.** No algoritmo de branch-and-price em si (relaxações dos nós, limitantes inferiores e podas), o Gurobi deve ser usado *apenas como solver de programação linear*: a árvore de enumeração, o branching e a geração de colunas são implementados por vocês. Há duas exceções, em que o Gurobi pode resolver problemas com variáveis inteiras:

   - no problema de pricing exato, caso vocês optem por uma formulação de programação linear inteira;
   - em uma heurística de arredondamento opcional (Seção 10.6).

5. **Implementação e execução.** O programa deve ser escrito em **C++** e receber os parâmetros abaixo na linha de comando:

parâmetro	significado
`-f <arquivo>`	arquivo de entrada com a instância
`-o <arquivo>`	arquivo de saída, onde a melhor solução é gravada
`-t <segundos>`	tempo máximo T, em segundos (tempo real)
`-r <semente>`	semente do gerador de números aleatórios (opcional; padrão 1)

   Por exemplo: `./binpack -f instancias/exemplo.txt -o saida.txt -t 60`.

Os requisitos gerais de execução são:

- o programa deve terminar em no máximo T segundos (mais o tempo de gravar a saída), devolvendo a melhor solução viável encontrada até então e o melhor limitante inferior que ele *provou* para OPT;
- se a árvore de enumeração for esgotada antes do tempo T, a solução devolvida é ótima, e o limitante inferior é igual ao seu valor;
- a solução gravada *deve ser viável*. O programa principal verifica isso antes de gravar; uma solução inviável é um erro;
- o arquivo de saída segue o formato da Seção 9.3.

10.2 O código fornecido

O diretório `projeto/` contém:

```
main.cpp          le a entrada, chama o solver, verifica e grava a saida
binpacksolver.cpp rotina SolveBinPacking(), com uma heuristica aleatorizada
binpacksolver.h   declaracao da rotina
binpacking.h      a classe BinPackingData (instancia + melhor solucao)
Makefile          make  ->  gera o executavel binpack
instancias/       a instancia do exemplo
```

O tipo `BinPackingData`. Uma única classe reúne toda a entrada (n, C, k, os tamanhos, as classes S_t e, para cada item, a lista das classes que o contêm), além do valor e da melhor solução encontrada até o momento (`best_value`, `best_solution`) e de um limitante inferior (`lower_bound`), inicialmente $\max\{\lceil s(I)/C \rceil, \max_t |S_t|\}$ (Fato 1). A classe também oferece `Conflict(i,j)`, `UpdateBest(...)` e `CheckSolution(...)`, que verifica se uma solução é viável.

A rotina `SolveBinPacking`. O programa principal lê a instância, guarda tudo em um objeto `BinPackingData` e chama a rotina `SolveBinPacking`, passando-lhe esse objeto e o tempo T. A rotina está no arquivo `binpacksolver.cpp`. Ao voltar, a solução em `data` é testada pelo programa principal. A rotina fornecida apenas executa a heurística descrita abaixo, para que o programa possa ser executado desde o início. Ela deve ser mantida como fonte da solução incumbente inicial e de colunas iniciais, e o algoritmo de branch-and-price deve ser implementado em seguida (`binpacksolver.cpp` e arquivos novos, se vocês quiserem).

A heurística fornecida. É uma versão probabilística do *First Fit Decreasing* que respeita os conflitos. Para cada item i, sejam

$$a_i = \frac{s_i}{C} \in (0,1], \qquad b_i = \frac{|\{t : \ i \in S_t\}|}{\max_{j \in I} |\{t : \ j \in S_t\}|} \in [0,1]$$

(e $b_i = 0$ se nenhum item pertence a alguma classe). A *prioridade* de i é dada por uma função de três parâmetros: o tamanho a, a participação b em classes e um número de *tendência* $\beta \in [0,1]$:

$$f(a,b,\beta) = \begin{cases} (1-2\beta)\,a + 2\beta\,r & \text{se } \beta \le \frac{1}{2}, \\ (2\beta-1)\,b + (2-2\beta)\,r & \text{se } \beta > \frac{1}{2}, \end{cases}$$

em que $r \in [0,1]$ é um número aleatório sorteado a cada avaliação. Assim:

- se $\beta = 0$, a prioridade é o tamanho, e os itens ficam em ordem decrescente de tamanho (First Fit Decreasing clássico);
- se β está perto de 1, itens com maior participação em classes de conflito aparecem primeiro;
- se $\beta = \frac{1}{2}$, a prioridade é totalmente aleatória: não depende do tamanho nem da participação em classes.

Em cada *rodada*, a heurística sorteia $\beta \in [0,1]$ uniformemente, calcula a prioridade de todos os itens com esse β, ordena os itens por prioridade decrescente (empates: maior tamanho primeiro) e os empacota em ordem, cada um no *primeiro* recipiente aberto em que cabe e em que não há item de uma classe em comum com ele. Se nenhum recipiente serve, abre-se um novo. A heurística executa até 1000 rodadas (ou até o tempo T, ou até atingir o limitante inferior) e mantém a melhor solução encontrada.

10.3 Formulação por conjuntos de itens

Seja $\mathcal{B}$ o conjunto de todos os recipientes válidos *não vazios* (Seção 3 e definição de recipiente válido). Para cada $B \in \mathcal{B}$ temos uma variável x_B, com $x_B = 1$ se, e somente se, B é um dos recipientes da solução:

$$(M) \qquad \min \sum_{B \in \mathcal{B}} x_B \quad \text{s.a.} \quad \sum_{B \in \mathcal{B}:\ i \in B} x_B = 1 \quad \forall i \in I, \qquad x_B \in \{0,1\}.$$

Fato 4. As soluções inteiras de (M) correspondem exatamente às soluções viáveis do problema, com o mesmo valor.

Demonstração. Seja x uma solução inteira de (M) e $\mathcal{B} = \{B : \ x_B = 1\}$. Cada item pertence a exatamente um recipiente de $\mathcal{B}$, de modo que $\mathcal{B}$ é uma partição de I em recipientes válidos, e o valor $\sum_B x_B$ é $|\mathcal{B}|$. Reciprocamente, uma solução viável $\mathcal{B}$ define $x_B = 1$ para $B \in \mathcal{B}$ e $x_B = 0$ para os demais. □

Como cada coluna é um *conjunto* de itens, não há simetria entre os recipientes: duas soluções que só diferem pela ordem dos recipientes são a mesma solução. O conjunto $\mathcal{B}$ tem tamanho exponencial; por isso trabalhamos com um subconjunto dele (Seção 10.4).

Seja z_{LP} o valor ótimo da relaxação linear de (M) (com $x_B \geq 0$). O dual da relaxação é

$$(D) \qquad \max \sum_{i \in I} u_i \quad \text{s.a.} \quad u(B) = \sum_{i \in B} u_i \leq 1 \quad \forall B \in \mathcal{B}, \qquad u_i \in \mathbb{R} \text{ (livres)},$$

com variáveis livres porque as restrições de (M) são igualdades.

Fato 5. $\lceil z_{\text{LP}} \rceil \leq \text{OPT}$ e $z_{\text{LP}} \geq \max\{\, s(I)/C, \ \max_t |S_t| \,\}$. Em particular, a relaxação linear é pelo menos tão forte quanto as cotas do Fato 1, exceto pelo arredondamento para cima.

Demonstração. A relaxação tem valor ótimo menor ou igual ao de (M), que é OPT (Fato 4), e OPT é inteiro. Seja x uma solução da relaxação. Então $C \sum_B x_B \geq \sum_B x_B \, s(B) = \sum_{i \in I} s_i \sum_{B \ni i} x_B = s(I)$. E, para cada classe S_t, $|B \cap S_t| \leq 1$ para todo B, de modo que $\sum_B x_B \geq \sum_B x_B |B \cap S_t| = \sum_{i \in S_t} \sum_{B \ni i} x_B = |S_t|$. □

10.4 Geração de colunas

Trabalhamos com um subconjunto $\mathcal{B}' \subseteq \mathcal{B}$ de colunas, o *problema mestre restrito* (PMR), cuja relaxação linear é resolvida pelo Gurobi como um PL.

Colunas iniciais. O PMR precisa começar viável. Uma escolha simples é colocar cada item sozinho em um recipiente (n colunas $\{i\}$, todas válidas pois $s_i \leq C$). As colunas dos recipientes da solução da heurística fornecida também devem ser incluídas, e a própria solução é a primeira *incumbente*.

Custo reduzido. Dadas as variáveis duais ótimas u do PMR, o custo reduzido de uma coluna B é

$$\bar{c}_B \ = \ 1 - u(B).$$

Uma coluna com $\bar{c}_B < 0$ pode melhorar a solução do PMR.

Problema de pricing. Queremos

$$\text{(Pricing)} \qquad \alpha^* \ = \ \max_{B \in \mathcal{B}} \ u(B),$$

e existe uma coluna de custo reduzido negativo se, e somente se, $\alpha^* > 1$. É um problema da *mochila com conflitos*: escolher itens de maior valor total u_i, com tamanho total no máximo C, sem dois itens de uma mesma classe. Ele é $\mathcal{NP}$-difícil, pois com $k = 0$ é o problema da mochila. Itens com $u_i \leq 0$ nunca ajudam (retirá-los de um recipiente válido o mantém válido e não diminui $u(B)$); eles podem ser ignorados.

Heurística primeiro, método exato depois. Em cada iteração:

1. execute uma *heurística* de pricing (por exemplo, um algoritmo guloso que considera os itens por ordem decrescente de u_i/s_i respeitando capacidade e conflitos, possivelmente com busca local ou aleatorização) e adicione ao PMR as colunas encontradas com $\bar{c}_B < 0$;
2. se a heurística não encontrou nenhuma, use o *método exato* para resolver o pricing. Se ele encontrar colunas de custo reduzido negativo, adicione-as ao PMR; caso contrário, a geração de colunas termina.

Limitante inferior. Seja z_{PMR} o valor ótimo do PL do PMR e u uma solução dual ótima.

Lema 6. (a) $z_{\text{PMR}} \geq z_{\text{LP}}$.

(b) (*Limitante de Farley.*) Se $\alpha \geq 1$ é tal que $u(B) \leq \alpha$ para todo $B \in \mathcal{B}$ (por exemplo, $\alpha = \max\{1, \alpha^*\}$), então $z_{\text{LP}} \geq z_{\text{PMR}}/\alpha$.

(c) Se nenhuma coluna tem custo reduzido negativo ($\alpha^* \leq 1$), então $z_{\text{PMR}} = z_{\text{LP}}$.

Demonstração. (a) O PMR é a relaxação linear de (M) restrita a um subconjunto das colunas, e o problema é de minimização. (b) Pela dualidade forte do PMR, $\sum_i u_i = z_{\text{PMR}}$. O vetor u/α satisfaz $u(B)/\alpha \leq 1$ para todo $B \in \mathcal{B}$, e portanto é viável para (D); pela dualidade fraca, $z_{\text{LP}} \geq \sum_i u_i/\alpha = z_{\text{PMR}}/\alpha$. (c) Com $\alpha = 1$, o item (b) dá $z_{\text{LP}} \geq z_{\text{PMR}}$, e o item (a) dá a igualdade. □

Portanto, o valor do PMR é só um limite *superior* para z_{LP}. Ele só se torna um limitante inferior para o nó quando o método exato *prova* que não existe coluna de custo reduzido negativo (item (c)), ou fornece uma cota α para o pricing (item (b)). A heurística, sozinha, não certifica nada. Como OPT é inteiro, o limitante de um nó pode ser arredondado para cima (com uma pequena tolerância numérica, por exemplo $\lceil z - 10^{-6} \rceil$). O item (b) é útil quando o tempo se esgota antes de a geração de colunas terminar: uma única chamada ao pricing exato dá um limitante válido.

10.5 Branching de Ryan–Foster

Suponha que a relaxação linear de (M) (com todas as colunas) tenha uma solução ótima x^*. Se x^* for inteira, o nó está resolvido: atualize a incumbente se for o caso e pode o nó. Caso contrário, é preciso ramificar. Ramificar diretamente em uma variável x_B é ruim: no ramo $x_B = 0$, o pricing voltaria a gerar a mesma coluna, e o ramo $x_B = 1$ é muito restritivo. O branching de Ryan–Foster ramifica sobre *pares de itens*.

Pares de itens. Para dois itens distintos i e j, defina

$$\varphi_{ij}(x) \;=\; \sum_{B \in \mathcal{B}:\; i,j \in B} x_B,$$

a fração de solução em que i e j estão *juntos* no mesmo recipiente.

Proposição 7. Se uma solução x^* da relaxação linear de (M) é fracionária, então existem itens $i \neq j$ com $0 < \varphi_{ij}(x^*) < 1$.

Demonstração. Como x^* é fracionária, existe B com $0 < x^*_B < 1$. Seja $i \in B$ (o conjunto B é não vazio). A restrição do item i é uma igualdade, de modo que existe outro conjunto $B' \neq B$ com $i \in B'$ e $x^*_{B'} > 0$ (pois $x^*_B < 1$). Como $B \neq B'$ e ambos contêm i, existe um item $j \neq i$ que pertence a exatamente um dos dois; suponha $j \in B \setminus B'$ (o outro caso é simétrico, trocando os papéis de B e B'). Então $\varphi_{ij} \geq x^*_B > 0$ e, como B' contém i mas não j,

$$\varphi_{ij} \;\leq\; \sum_{B'' \ni i} x^*_{B''} - x^*_{B'} \;=\; 1 - x^*_{B'} \;<\; 1. \qquad \square$$

Em outras palavras, se todos os φ_{ij} são inteiros, então x^* é inteira.

Os dois ramos. Escolhido um par $\{i, j\}$ com φ_{ij} fracionário, criamos dois filhos:

ramo $\varphi_{ij} = 1$ (“juntos”)	ramo $\varphi_{ij} = 0$ (“separados”)
i e j ficam *no mesmo recipiente*	i e j ficam *em recipientes diferentes*

Cada ramo é imposto *removendo do PMR as colunas incompatíveis* e *proibindo o pricing de gerá-las novamente*:

- juntos: remova as colunas que contêm *exatamente um* dos dois itens. O pricing só pode gerar recipientes que contêm ambos ou nenhum;
- separados: remova as colunas que contêm *os dois* itens. O pricing só pode gerar recipientes que não contêm ambos. Isto equivale a acrescentar o par $\{i, j\}$ ao grafo de conflitos ("uma classe com dois itens").

Lema 8. Toda solução viável do nó que satisfaz as decisões do nó satisfaz as decisões de exatamente um dos dois filhos, e o conjunto de soluções de cada filho está contido no do nó. Portanto, o menor valor de uma solução do nó é o menor entre os dos dois filhos.

Demonstração. Em uma solução viável, i e j estão no mesmo recipiente ou não, e não ocorrem os dois. No primeiro caso, nenhum recipiente contém exatamente um deles, de modo que a solução não usa as colunas removidas no filho "juntos". No segundo, nenhum recipiente contém os dois, e a solução não usa as colunas removidas no filho "separados". Como cada filho só remove colunas, suas soluções já eram soluções do nó. □

Cada nó é uma instância menor do mesmo problema. As decisões de um nó são um conjunto T de pares "juntos" e um conjunto D de pares "separados". Sejam os *grupos* as componentes conexas do grafo (I, T), com tamanho $s(G) = \sum_{i \in G} s_i$. Dois grupos distintos G, G' *conflitam* se existem $i \in G$ e $j \in G'$ que pertencem a uma classe comum ou tais que $\{i, j\} \in D$. Um grupo G é *inválido* se $s(G) > C$ ou se contém dois itens distintos que pertencem a uma classe comum ou formam um par de D.

Lema 9. (a) Se algum grupo é inválido, nenhuma solução viável satisfaz as decisões do nó, e o nó deve ser podado.

(b) Caso contrário, os grupos, com os tamanhos $s(G)$ e os conflitos acima, formam uma instância I_N do mesmo problema, e a aplicação $\{G_1, \ldots, G_r\} \mapsto G_1 \cup \cdots \cup G_r$ é uma bijeção entre as soluções viáveis de I_N e as soluções viáveis do problema original que satisfazem as decisões do nó, preservando o valor.

Demonstração. (a) Um grupo inválido teria de estar inteiro em um único recipiente (pois seus itens são ligados por pares "juntos"), que não seria válido. (b) Em uma solução que satisfaz T, cada grupo está inteiro em um recipiente, de modo que cada recipiente é uma união de grupos. Uma união de grupos é um recipiente válido do problema original, e que respeita D, se, e somente se, o seu conjunto de grupos tem tamanho total no máximo C e não tem dois grupos que conflitam. Ou seja, se, e somente se, é um recipiente válido de I_N. Todo grupo tem tamanho no máximo C, pois não é inválido, de modo que I_N é uma instância legítima. □

Em consequência:

- o PMR de um nó é o problema (M) da instância I_N, com *um item por grupo*. As colunas de um nó são as uniões de grupos que formam um recipiente válido de I_N;
- a Proposição 7 vale para I_N: se a solução do nó é fracionária, existe um par de *grupos* G, G' com $0 < \varphi_{GG'} < 1$. Ramificar nele equivale a ramificar sobre quaisquer $i \in G$ e $j \in G'$. No ramo "juntos", os dois grupos são fundidos (e o filho é podado se o grupo resultante é inválido); no ramo "separados", os dois grupos passam a conflitar;
- o pricing de um nó é a mochila com conflitos sobre os grupos, com valores u_G e tamanhos $s(G)$;
- a coluna $\{G\}$ de cada grupo é um recipiente válido de I_N (se N não é podado). Portanto, o PMR de qualquer nó pode ser inicializado com essas colunas e é sempre viável: *não são necessárias variáveis artificiais.*

Vocês podem implementar isso de duas formas: criar explicitamente a instância I_N em cada nó, ou manter um banco global de colunas (conjuntos de itens originais) e, em cada nó, usar apenas as colunas *admissíveis*, isto é, as que contêm cada grupo inteiro ou nenhum item dele, e nenhum par de D. Em ambos os casos, para cada conjunto de itens guardem uma única coluna.

Árvore de enumeração. A escolha do par a ramificar (por exemplo, o par com φ mais próximo de $\frac{1}{2}$) e a ordem de exploração dos nós (por exemplo, busca em profundidade ou melhor limitante) ficam a cargo de vocês. O limitante de um filho nunca é menor que o do pai (o filho só tem menos soluções). Um nó é podado se o seu limitante inferior (obtido apenas após o pricing exato certificar a otimalidade, ou pelo Lema 6(b)), arredondado para cima, não é menor que o valor da melhor solução conhecida. O limitante inferior global que o programa grava é o menor limitante entre os nós ainda abertos (e é igual ao valor da incumbente se a árvore foi esgotada), nunca menor que o limitante inicial de `lower_bound`.

10.6 Heurística de arredondamento (opcional)

Para obter boas soluções antes de provar a otimalidade, vocês podem, em algum nó (por exemplo, no nó raiz), pedir ao Gurobi a melhor solução inteira do problema (M) restrito a um conjunto de colunas já geradas (todas, ou um subconjunto delas), com um pequeno limite de tempo. Qualquer coluna válida já gerada pode ser usada, mesmo aquelas removidas do PMR do nó atual por causa do branching: o resultado é uma solução viável do problema *original*. O valor obtido serve *apenas* como limitante superior (incumbente); ele nunca é usado como limitante inferior. O tempo gasto nesta heurística conta dentro de T. Antes de aceitá-la, verifique a solução (`CheckSolution`).

10.7 Resumo do algoritmo

1. Execute a heurística fornecida: obtenha a incumbente e colunas iniciais, e acrescente as colunas $\{i\}$ de cada item.
2. Para cada nó da árvore:
   (a) resolva o PL do PMR com o Gurobi;
   (b) faça o pricing, primeiro por heurística e depois pelo método exato, e repita até que o exato não encontre coluna de custo reduzido negativo;
   (c) se o limitante inferior (arredondado para cima) for pelo menos o valor da incumbente, pode o nó;
   (d) se a solução do PL for inteira, atualize a incumbente e pode o nó;
   (e) caso contrário, escolha um par de itens (ou de grupos) com φ fracionário (Proposição 7) e crie os dois filhos ("juntos" e "separados").
3. Ao final (árvore esgotada ou tempo T), grave a melhor solução e o melhor limitante inferior.

11 Entrega e uso de ferramentas de IA

O projeto é **individual**. O objetivo é que cada aluno *domine* o algoritmo e o código que entrega, e saiba verificar que tudo funciona.

11.1 Uso de ferramentas de IA

O uso de ferramentas de IA (por exemplo, ChatGPT e Claude) é **permitido**, sob as condições abaixo.

- **Transparência.** Todas as *conversas* com ferramentas de IA devem ser entregues completas, em ordem cronológica, com os prompts e as respostas (e não apenas os prompts).

- **Responsabilidade.** Você é responsável por todo o código e todo o texto entregues, como se fossem seus, e deve conseguir explicar e modificar qualquer parte deles.
- **Individualidade.** Não é permitido copiar código ou texto de outros alunos, nem compartilhar o seu. Isso inclui as suas conversas com ferramentas de IA.

11.2 O que entregar

A entrega é feita **por e-mail**, para `fkm@unicamp.br`, com o assunto (*subject*) exatamente igual a

`[PLI] Bin Packing com Classes de Conflito`

e com um único arquivo compactado (`.zip` ou `.tar.gz`) anexado, com a estrutura:

```
codigo/   fontes em C++ e Makefile (obrigatorio)
testes/   instancias de teste e scripts
ia/       conversas completas com ferramentas de IA, em ordem cronologica
README    como compilar e executar, versoes usadas, metodos de pricing
```

Regras sobre os arquivos.

- O arquivo enviado **não deve conter código executável nem código objeto**: nada de executáveis, arquivos `.o`, bibliotecas (`.a`, `.so`), nem diretórios de compilação. Envie apenas o código-fonte, os scripts e os demais arquivos de texto, PDFs e instâncias de teste.
- A entrega **deve ter um** `Makefile` em `codigo/`: o comando `make`, executado nesse diretório, deve compilar o programa e gerar o executável. O caminho de instalação do Gurobi deve ser configurável no `Makefile` (por exemplo, por uma variável no início do arquivo).

Prazo. A entrega deve ser feita em até **duas semanas** a partir da data de divulgação deste enunciado.